

파이썬을 이용한 알기쉬운 수치해석

김시조

국립경국대학교: (구)국립안동대학교

February 17, 2026

머릿말

Note. 참고 URL

https://github.com/seejokim1/NA_with_python/tree/main

21세기는 기술 혁신과 지식의 융합이 핵심 동력으로 작용하는 시대다. 특히 4차 산업혁명과 인공지능 기술의 발전은 전통적인 공학 분야에 새로운 도전과 기회를 제공하고 있다. 이러한 환경에서 수치해석은 공학 문제를 해결하는 핵심 도구로, 다양한 데이터 기반 분석과 최적화 문제 해결에 활용되고 있다.

현재 수치해석의 중요성은 더욱 강조되고 있으며, 특히 자율주행 자동차의 수치해석에서 가장 중요한 것은 정확도와 실시간 처리이다. 자율주행 차량은 다양한 센서 데이터를 실시간으로 처리하며, 도로 상황을 정확하게 예측하고 반응해야 하기 때문이다. 이러한 기술은 수치해석을 통해 더욱 정교해지고, 안전한 자율주행을 가능하게 한다.

또한 AI와 수치해석은 깊은 관계를 맺고 있으며, AI의 발전이 수치해석 방법론에 많은 영향을 미쳤고, 반대로 수치해석 기술이 AI의 성능을 향상시키는 데 중요한 역할을 해왔다. 수치해석은 실세계의 문제를 수학적으로 모델링하고 해결하기 위한 다양한 알고리즘과 방법을 제공하며, AI는 이러한 수학적 모델을 데이터 기반으로 학습하고 최적화하는 능력을 갖추고 있다. 이 둘의 결합은 현대 과학과 공학의 중요한 연구 분야로 자리 잡았다.

예를 들어, 2024년 노벨 물리학상은 머신러닝과 AI 기술이 물리학 문제 해결에 중요한 기여를 한 연구자에게 수여되었다. 이는 머신러닝이 복잡한 물리적 시스템을 시뮬레이션하거나 예측하는 데 사용되고 있음을 보여준다. 또한 알파고에 이어 알파폴드와 같은 AI 기술이 노벨 화학상을 수상하며, 수치해석 기법을 바탕으로 AI 모델이 실세계 문제를 해결하는 데 어떻게 활용될 수 있는지를 보여주었다.

이 책은 파이썬 프로그래밍 언어를 활용하여 수치해석의 기초부터 심화 내용까지 단계적으로 학습할 수 있도록 구성되었다. 첫 장에서는 파이썬의 기본 문법과 데이터 처리에 필수적인 라이브러리(예: NumPy, Matplotlib)를 다루며, 프로그래밍과 수치해석의 연결고리를 제공한다. 이어지는 장에서는 비선형 방정식 근사법, 선형 연립방정식 해법, 수치미분 및 수치적분, 보간법, 수치 회귀 등 전통적인 수치해석의 핵심 주제들을 체계적으로 소개한다.

특히 후반부에서는 상미분 방정식과 편미분 방정식의 수치적 해법을 다루며, 유한차분법(Finite Difference Method)과 유한요소법(Finite Element Method)의 응용을 통해 실질적인 문제 해결 방법을 제시한다. 이와 더불어 Python을 활용하여 다양한 수치해석 문제를 직접 해결해보는 연습문제를 포함해 독자가 이론과 실습을 병행하며 학습할 수 있도록 하였다.

김시조

Contents

1	파이썬의 이해	1
2	비선형 방정식 구하기	2
3	선형연립방정식 수치해법	3
3.1	선형연립방정식 수치해법 개요	3
3.1.1	스칼라(Scalar), 벡터(Vector), 행렬(Matrix)의 관점에서의 중요성	3
3.1.2	선형연립방정식들의 해 존재	4
3.1.3	행렬의 선형변환	5
3.1.4	해 존재를 위한 행렬의 행공간(row space)과 영공간(null space)	6
3.1.5	선형연립방정식의 수치해법 개요	7
3.2	가우스 소거법	9
3.2.1	(3.2.1) 가우스 소거법 개요	9
3.2.2	전진 소거 (Forward Elimination)	11
3.2.3	전진소거 알고리즘	12
3.2.4	후진 대입 (Backward Substitution)	13
3.2.5	가우스 소거법 전체 코드 함수 구현	13
3.2.6	수치적 안정성을 위한 부분 피벗팅	14
3.3	LU Decomposition Method	15
3.3.1	LU 분해법 개요 (Doolittle LU 분해법)	15
3.3.2	알고리즘의 원리	17
3.3.3	$A = LU$ 증명	18
3.3.4	Doolittle LU 분해법 공식 유도 과정 이용해서 손으로 풀기	20
3.3.5	Doolittle LU Python 코드 예제	22
3.3.6	(3.3.6) Crout LU 분해법	22
3.3.7	$[A] = [L][D][L]^T$ 분해 및 Cholesky LU 분해	24
3.3.8	SciPy에서의 LU 분해	25
3.3.9	SciPy에서 LU 분해를 활용한 선형 방정식 풀이	26
3.3.10	LU 분해법과 NumPy의 <code>numpy.linalg.solve</code> 비교	26
3.4	Jacobi, Gauss-Seidel, SOR 반복법	27
3.4.1	Jacobi, Gauss-Seidel, SOR 반복법 개요	27
3.4.2	Jacobi 반복법 (핵심 정리)	28
3.4.3	Gauss-Seidel 반복법 (핵심 정리)	30
3.4.4	SOR (Successive Over-Relaxation) 반복법	32
3.5	연습문제	33

Chapter 1

파이썬의 이해

Chapter 2

비선형 방정식 구하기

Chapter 3

선형연립방정식 수치해법

3.1 선형연립방정식 수치해법 개요

3.1.1 스칼라(Scalar), 벡터(Vector), 행렬(Matrix)의 관점에서의 중요성

스칼라, 벡터, 행렬은 공학 및 수학에서 가장 기본이 되는 데이터 표현 방식이다. 이들은 서로 포함 관계를 가지며, 구조와 역할이 다르다.

① 스칼라 (Scalar)

- 하나의 수로 구성된 데이터
- 크기만 가지며 방향은 없음
- 예: 5, -3.4 , π

② 벡터 (Vector)

- 여러 개의 스칼라로 구성된 1차원 배열
- 크기와 방향을 동시에 표현
- 행 벡터 (row vector) 또는 열 벡터 (column vector) 형태

$$\mathbf{v} = \begin{bmatrix} v_1 \\ v_2 \\ v_3 \end{bmatrix}$$

- 벡터는 스칼라를 원소로 갖는 구조

③ 행렬 (Matrix)

- 벡터들이 모여 구성된 2차원 배열

- 행(row)과 열(column)로 구성

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 6 & 7 \\ 2 & 4 & 8 \end{bmatrix}$$

- 여러 개의 벡터를 동시에 표현 가능

구조적 관계

$$\text{Scalar} \subset \text{Vector} \subset \text{Matrix}$$

- 스칼라는 벡터의 원소
- 벡터는 행렬의 한 행 또는 한 열
- 행렬은 벡터들의 집합

공학적 중요성

- 선형연립방정식 표현: $Ax = b$
- 물리 시스템 모델링
- 데이터 분석 및 인공지능 계산
- 수치해석 알고리즘의 기본 구조

3.1.2 선형연립방정식들의 해 존재

이 절에서는 선형연립방정식의 해의 존재성과 유일성을 판단하는 논리적 흐름을 정리한다.

선형 시스템

$$Ax = b$$

에 대해 다음을 확인한다.

판단 절차

1. 먼저 해가 존재하는지(Exist) 확인한다.
 - 해가 존재하지 않으면 (NO) → 해 없음.
2. 해가 존재한다면, 유일한지(Unique) 여부를 확인한다.
 - YES → 유일해 (Unique Solution)
 - NO → 해가 2개 이상 존재 (무한해 가능성 포함)

대부분의 수치해석 방법 (가우스 소거법, LU 분해, 역행렬 계산 등)은 해가 존재하며 유일하다고 가정하고 계산을 수행한다. 따라서 실제 계산 전에 해의 존재성과 유일성을 이론적으로 검토하는 과정이 중요하다.

```
[ node distance=2cm, >=Stealth ]
(exist) [rectangle,draw] Exist?; (yes1) [right=2.5cm of exist] YES!; (unique)
[diamond,draw,aspect=2,below=1.5cm of yes1] Unique?; (yes2) [below=1.5cm of
unique] YES; (no2) [right=3cm of unique, align=center] NO
Sol ≥ 2;
(no1) [below=1.5cm of exist] NO;
[->] (exist) - (yes1); [->] (exist) - (no1); [->] (yes1) - (unique); [->] (unique) - (yes2);
[->] (unique) - (no2);
```

핵심 정리

- 해가 존재하지 않는 경우 → No Solution
- 해가 유일한 경우 → Unique Solution
- 해가 여러 개인 경우 → Infinite Solutions 가능
- 선형대수에서 랭크(rank) 조건이 핵심 기준이 된다.

3.1.3 행렬의 선형변환

1. 행렬의 선형변환은?

벡터 공간에서 축의 스케일링, 회전, 반사, 전단 등을 포함한 변환을 수학적으로 설명하며, 이는 다양한 물리적, 공학적 시스템을 모델링하는 데 필수적이다.

2. 선형변환을 이해하면?

복잡한 시스템의 구조를 단순화하거나 최적화할 수 있으며, 특히 고유값과 고유벡터는 변환의 축을 파악하는 데 중요한 도구이다.

3. 예제:

빨간색 벡터와 녹색 벡터는 행렬 $[A]$ 에 의해 변환된 기본 벡터를 나타낸다. 왼쪽 그래프는 원래의 기준 좌표계를 나타내며, 이 벡터들은 변환 전에 단위 벡터로 시작하여 원점에서 시작해 각 방향을 나타낸다.

오른쪽 그래프는 행렬 A 에 의해 변환된 후의 좌표계를 나타낸다. 변환 후 좌표계는 기울어진 형태를 나타내며, 원래의 직교 좌표계를 변경한다.

이러한 변환은 선형변환의 주요 특징인 방향 및 크기 변화를 시각적으로 보여준다.

실습예제: https://github.com/seejokim1/NA_with_python/blob/main/chapter03/section_03_01_03_linear_transformation.py

3.1.4 해 존재를 위한 행렬의 행공간(row space)과 영공간(null space)

아래와 같은 선형연립방정식의 해 공간을 구해보자.

$$\begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 8 \\ 4 \end{bmatrix} \quad (3.1.4-1)$$

행렬을 가우스 소거하여 Reduced Row Echelon Form으로 만들면

$$\left[\begin{array}{cc|c} 1 & 2 & 8 \\ 2 & 4 & 4 \end{array} \right] \longrightarrow \left[\begin{array}{cc|c} 1 & 2 & 4 \\ 0 & 0 & 0 \end{array} \right] \quad (3.1.4-2)$$

따라서

$$x_1 + 2x_2 = 4 \quad (3.1.4-3)$$

여기서 자유변수 $x_2 = t$ 로 두면

$$x_1 = 4 - 2t$$

따라서 해는

$$\begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 4 \\ 0 \end{bmatrix} + t \begin{bmatrix} -2 \\ 1 \end{bmatrix} \quad (3.1.4-4)$$

영공간 (Null Space)

영공간은 다음을 만족하는 벡터들의 집합이다.

$$A\mathbf{x} = 0$$

위 행렬의 경우,

$$\mathbf{n} = \begin{bmatrix} -2 \\ 1 \end{bmatrix}$$

은 영공간 벡터이다.

$$\text{Null}(A) = \text{span} \left\{ \begin{bmatrix} -2 \\ 1 \end{bmatrix} \right\} \quad (3.1.4-5)$$

행공간 (Row Space)

행공간은 행벡터들이 생성하는 공간이다.

$$\text{Row}(A) = \text{span} \{ [1 \ 2] \} \quad (3.1.4-6)$$

행공간과 영공간은 서로 직교한다.

$$\begin{bmatrix} 1 & 2 \end{bmatrix} \cdot \begin{bmatrix} -2 \\ 1 \end{bmatrix} = -2 + 2 = 0$$

따라서

$$\text{Row}(A) \perp \text{Null}(A)$$

핵심 개념 정리

- 해가 존재하면 일반해는

$$\text{특정해} + c \times \text{영공간 벡터}$$

- 영공간 벡터는 자유도를 나타낸다.
- 행공간과 영공간은 서로 직교한다.
- 랭크(rank)가 해 존재와 유일성을 결정한다.

실습예제: https://github.com/seejokim1/NA_with_python/blob/main/chapter03/section_03_01_04_row_null_space.py

3.1.5 선형연립방정식의 수치해법 개요

n 개의 미지수를 갖는 n 개의 선형연립방정식

$$A\mathbf{x} = \mathbf{b}$$

은 행렬이 가역일 경우 다음과 같이 표현할 수 있다.

$$\mathbf{x} = A^{-1}\mathbf{b} \quad (3.1.5-1)$$

그러나 대규모 선형 시스템의 경우 A^{-1} 을 직접 계산하는 것은 계산량이 매우 크며 비효율적이다. 따라서 실제 계산에서는 역행렬을 구하지 않고 직접 해를 구하는 수치적 방법을 사용한다.

① 가우스 소거법 (Gaussian Elimination)

연립방정식을 계단행렬 형태로 변환한 후 후진 대입을 통해 해를 구하는 가장 기본적인 직접법이다.

- 계산 복잡도: $\mathcal{O}(n^3)$
- 안정성과 효율성이 비교적 우수
- 피벗팅(pivoting)을 하지 않으면 수치 오차가 커질 수 있음

② LU 분해법 (LU Decomposition)

계수 행렬 A 를

$$A = LU$$

로 분해하여 해를 구하는 방법이다.

- L : 하삼각행렬
- U : 상삼각행렬
- 여러 개의 b 에 대해 반복 계산 시 효율적

해는 다음 순서로 구한다.

$$Ly = b \Rightarrow Ux = y$$

③ 반복법 (Iterative Methods)

대규모 시스템에서 주로 사용되며, 해를 점진적으로 개선해 나가는 방법이다.

- Jacobi Method
- Gauss-Seidel Method
- SOR (Successive Over-Relaxation)

Jacobi 방법은 이전 단계의 값을 이용하여 모든 변수를 동시에 갱신한다. Gauss-Seidel 방법은 갱신된 값을 즉시 반영하여 수렴 속도를 개선한다. SOR 방법은 완화계수 ω 를 도입하여 수렴 속도를 더욱 향상시킨다.

$$x^{(k+1)} = (1 - \omega)x^{(k)} + \omega\tilde{x}^{(k+1)}$$

여기서 $\omega = 1$ 이면 Gauss-Seidel 방법과 동일하며, $\omega > 1$ 은 과완화(over-relaxation)를 의미한다.

핵심 개념 정리

- 역행렬 계산은 일반적으로 비효율적이다.
- 직접법(Direct Methods)은 정확도가 높으나 계산량이 크다.
- 반복법(Iterative Methods)은 대규모 시스템에 적합하다.
- 문제의 크기와 행렬의 구조에 따라 적절한 방법을 선택해야 한다.
- 수치적 안정성을 위해 피벗팅과 조건수(condition number)를 고려해야 한다.

3.2 가우스 소거법

3.2.1 (3.2.1) 가우스 소거법 개요

① 소거법의 의미

가우스 소거법(Gaussian Elimination)은 연립방정식을 단계적으로 변형하여 상삼각 행렬 형태로 만든 뒤 해를 구하는 방법이다.

크게 두 단계로 구성된다.

- 전진 소거 (Forward Elimination)
- 후진 대입 (Back Substitution)

선형연립방정식의 일반형

n 개의 미지수를 갖는 n 개의 선형연립방정식은

$$[A]\mathbf{x} = \mathbf{b}$$

형태로 표현된다.

성분 형태로 쓰면 다음과 같다.

$$\begin{aligned} a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n &= b_1 \\ a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n &= b_2 \\ &\vdots \\ a_{n1}x_1 + a_{n2}x_2 + \cdots + a_{nn}x_n &= b_n \end{aligned} \quad (3.2.1-1)$$

Index 표기법

위 식은 인덱스 표기법으로 다음과 같이 표현할 수 있다.

$$\sum_{j=1}^n a_{ij}x_j = b_i \quad (1 \leq i \leq n) \quad (3.2.1-2)$$

이는 i 번째 방정식에서 모든 미지수 x_j 에 대한 계수의 합을 의미한다.

전진 소거 (Forward Elimination)

행 연산을 이용하여 행렬 $[A]$ 를 상삼각 행렬로 변환한다.

$$[A]\mathbf{x} = \mathbf{b} \longrightarrow [U]\mathbf{x} = \mathbf{b}'$$

여기서 $[U]$ 는 다음과 같은 상삼각 행렬이다.

$$U = \begin{bmatrix} u_{11} & u_{12} & u_{13} & \cdots & u_{1n} \\ 0 & u_{22} & u_{23} & \cdots & u_{2n} \\ 0 & 0 & u_{33} & \cdots & u_{3n} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & u_{nn} \end{bmatrix} \quad (3.2.1-3)$$

핵심은 피벗(pivot)을 기준으로 아래 행의 원소를 0으로 만드는 것이다.

후진 대입 (Back Substitution)

상삼각 행렬이 되면 마지막 방정식부터 역순으로 해를 구한다.

$$u_{nn}x_n = b_n$$

$$x_n = \frac{b_n}{u_{nn}}$$

이를 위로 거슬러 올라가며

$$x_{n-1}, x_{n-2}, \dots, x_1$$

을 순차적으로 계산한다.

가우스 소거법의 핵심 특징

- 연립방정식을 행 연산으로 변환
- 상삼각 행렬로 구조 단순화
- 계산 복잡도: $O(n^3)$
- 피벗 선택이 수치적 안정성에 매우 중요

3.2.2 전진 소거 (Forward Elimination)

아래는 선형 연립방정식을 n 개의 미지수와 n 식으로 표현한 것이다.

$$\begin{aligned} a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n &= b_1 \\ a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n &= b_2 \\ &\vdots \\ a_{n1}x_1 + a_{n2}x_2 + \cdots + a_{nn}x_n &= b_n \end{aligned} \quad (3.2.2-1)$$

① 피벗 방정식

가우스 소거법에서 피벗(pivot)은 기준이 되는 계수를 의미한다. 첫 번째 단계에서는 a_{11} 을 피벗으로 사용한다.

② 첫 번째 피벗을 이용하여 x_1 소거

두 번째 방정식에서 x_1 을 제거하기 위해 다음과 같은 소거 계수(factor)를 사용한다.

$$\frac{a_{21}}{a_{11}}$$

두 번째 방정식은 다음과 같이 변환된다.

$$\begin{aligned} a'_{2j} &= a_{2j} - \frac{a_{21}}{a_{11}}a_{1j} \\ b'_2 &= b_2 - \frac{a_{21}}{a_{11}}b_1 \end{aligned} \quad (3.2.2-2)$$

일반적으로,

$$\begin{aligned} a'_{ij} &= a_{ij} - \frac{a_{i1}}{a_{11}}a_{1j}, & b'_i &= b_i - \frac{a_{i1}}{a_{11}}b_1 \\ (2 \leq i \leq n), & (2 \leq j \leq n) \end{aligned} \quad (3.2.2-3)$$

이를 통해 첫 번째 열 아래의 원소가 모두 0이 된다.

③ 두 번째 피벗을 이용하여 x_2 소거

이제 두 번째 피벗 a'_{22} 을 사용한다.

소거 계수는

$$\frac{a'_{32}}{a'_{22}}$$

일반식은 다음과 같다.

$$\begin{aligned} a''_{ij} &= a'_{ij} - \frac{a'_{i2}}{a'_{22}}a'_{2j}, & b''_i &= b'_i - \frac{a'_{i2}}{a'_{22}}b'_2 \\ (3 \leq i \leq n), & (3 \leq j \leq n) \end{aligned} \quad (3.2.2-4)$$

④ $(n-1)$ 번째 피벗까지 반복

이 과정을 반복하면 상삼각 행렬 형태가 된다.

$$\begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ 0 & a'_{22} & \cdots & a'_{2n} \\ 0 & 0 & \ddots & \vdots \\ 0 & 0 & 0 & a_{nn}^{(n-1)} \end{bmatrix}$$

즉,

$$[A]x = b \longrightarrow [U]x = \tilde{b} \quad (3.2.2-5)$$

여기서 U 는 상삼각 행렬(Upper Triangular Matrix)이다.

이렇게 전진 소거를 통해 모든 하부 삼각 원소를 0으로 만들면 후진 대입(Back Substitution)을 통해 해를 계산할 수 있다.

3.2.3 전진소거 알고리즘

여기서

$$\frac{a_{ik}}{a_{kk}}$$

를 피벗(pivot)을 기준으로 한 소거계수(factor)라 하고, 이는 프로그램 및 추후 $[L][U]$ 분해법에서도 사용되는 중요한 계수이다.

전진소거 알고리즘은 아래와 같다. 실제 프로그램에서는

$$\frac{a_{ik}}{a_{kk}}$$

을 소거계수(factor)로 치환하여 작성한다.

$$\text{for}(2 \leq k \leq n-1),$$

$$\text{for}(k+1 \leq i \leq n),$$

$$b'_i \leftarrow b_i - \frac{a_{ik}}{a_{kk}} b_k \quad (3.2.3-1)$$

$$\text{for}(k \leq j \leq n)$$

$$a'_{ij} \leftarrow a_{ij} - \frac{a_{ik}}{a_{kk}} a_{kj}$$

실습예제: https://github.com/seejokim1/NA_with_python/blob/main/chapter03/section_03_02_03_forward_elimination.py

3.2.4 후진 대입 (Backward Substitution)

Back Substitution은 행상다리꼴 형태의 행렬을 만든 뒤 방정식의 해를 구하는 단계이다. 가장 아래 행의 방정식의 해를 구하고 이를 위에서부터 대입하여 모든 미지수에 대한 해를 도출한다.

수식 정리는 다음과 같은 형태로 일반화할 수 있다. 이를 수식으로 정리하면 아래와 같다.

$$\begin{aligned}
 a_{11}x_1 + a_{12}x_2 + a_{13}x_3 + \cdots + a_{1n}x_n &= b_1 \\
 0x_1 + a_{22}x_2 + a_{23}x_3 + \cdots + a_{2n}x_n &= b_2 \\
 &\vdots \\
 0x_1 + 0x_2 + \cdots + a_{n-1,n-1}x_{n-1} + a_{n-1,n}x_n &= b_{n-1} \\
 0x_1 + 0x_2 + 0x_3 + \cdots + a_{nn}x_n &= b_n
 \end{aligned} \tag{3.2.4-1}$$

마지막 식으로부터

$$x_n = \frac{b_n}{a_{nn}}$$

이를 위 식에 대입하면

$$x_{n-1} = \frac{1}{a_{n-1,n-1}} (b_{n-1} - a_{n-1,n}x_n)$$

일반적으로는

$$x_i = \frac{1}{a_{ii}} \left(b_i - \sum_{j=i+1}^n a_{ij}x_j \right), \quad i = n-1, n-2, \dots, 1$$

실습예제: https://github.com/seejokim1/NA_with_python/blob/main/chapter03/section_03_02_04_back_substitution.py

3.2.5 가우스 소거법 전체 코드 함수 구현

실습예제: https://github.com/seejokim1/NA_with_python/blob/main/chapter03/section_03_02_05_gaussian_elimination_full.py

```
import numpy as np

def gauss_elimination(A, b):
    n = len(b)

    # Forward elimination
```



```

for k in range(n-1):
    for i in range(k+1, n):
        factor = A[i][k] / A[k][k]
        A[i][k:] = A[i][k:] - factor * A[k][k:]
        b[i] = b[i] - factor * b[k]

# Back substitution
x = np.zeros(n)
for i in range(n-1, -1, -1):
    x[i] = (b[i] - np.dot(A[i][i+1:], x[i+1:])) / A[i][i]

return x

```

3.2.6 수치적 안정성을 위한 부분 피벗팅

가우스 소거법에서 계산의 수치적 안정성을 높이기 위해 **부분 피벗팅(Partial Pivoting)**을 사용한다.

왜 피벗팅이 필요한가?

- 피벗 요소 a_{kk} 가 매우 작으면

$$\frac{a_{ik}}{a_{kk}}$$

이 매우 커져 수치 오차가 증폭된다.

- 반올림 오차(round-off error)가 누적되어 해의 정확도가 크게 떨어질 수 있다.

부분 피벗팅(Partial Pivoting)의 원리

- 현재 열에서 절댓값이 가장 큰 원소를 피벗으로 선택한다.
- 해당 행과 현재 피벗 행을 서로 교환한다.
- 이후 일반 가우스 소거를 진행한다.

즉,

$$|a_{pk}| = \max_{i \geq k} |a_{ik}|$$

인 행 p 를 찾아 행을 교환한다.

가우스 소거 + 부분 피벗팅 절차

1. 각 단계 k 에서 피벗 열의 최대 절댓값 행을 선택
2. 현재 행과 교환
3. 소거 수행 (Forward Elimination)
4. 후진 대입 (Backward Substitution)

수치적 효과

- 반올림 오차 감소
- 불안정한 계산 방지
- 실제 계산에서 표준적으로 사용됨

예시 (불안정한 시스템)

$$10^{-20}x + y = 1$$

$$x + y =$$

실습예제: https://github.com/seejokim1/NA_with_python/blob/main/chapter03/section_03_02_06_partial_pivoting.py

```
def partial_pivot(A, b, k):
    n = len(b)
    max_index = max(range(k, n), key=lambda i: abs(A[i][k]))
    if k != max_index:
        A[[k, max_index]] = A[[max_index, k]]
        b[[k, max_index]] = b[[max_index, k]]
```

3.3 LU Decomposition Method

3.3.1 LU 분해법 개요 (Doolittle LU 분해법)

LU 분해의 기본 개념

선형연립방정식

$$Ax = b$$

에서 계수 행렬 A 를

$$A = LU$$

로 분해하여 계산을 단순화하는 방법이다.

Doolittle LU 분해의 구조

$$A = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{bmatrix}$$

$$L = \begin{bmatrix} 1 & 0 & \cdots & 0 \\ \ell_{21} & 1 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ \ell_{n1} & \ell_{n2} & \cdots & 1 \end{bmatrix}$$

$$U = \begin{bmatrix} u_{11} & u_{12} & \cdots & u_{1n} \\ 0 & u_{22} & \cdots & u_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & u_{nn} \end{bmatrix}$$

- L : 대각 성분이 1인 하삼각행렬
- U : 상삼각행렬

LU 분해의 계산 절차

1. $A = LU$ 조건을 이용해 행렬 원소 비교
2. U 의 위쪽 성분 계산
3. L 의 아래쪽 성분 계산

즉,

$$a_{ij} = \sum_{k=1}^{\min(i,j)} \ell_{ik} u_{kj}$$

을 만족하도록 원소를 결정한다.

LU 분해의 장점

- 여러 개의 b 에 대해 반복 계산 시 효율적
- 가우스 소거 과정을 체계적으로 정리한 형태
- 계산 복잡도: $O(n^3)$

선형 시스템 풀이 과정

$$Ax = b \quad \Rightarrow \quad LUx = b$$

$$Ly = b \quad (\text{Forward Substitution})$$

$$Ux = y \quad (\text{Backward Substitution})$$

3.3.2 알고리즘의 원리

1. LU 분해의 기본 개념

LU 분해의 핵심은 정방행렬 A 를 다음과 같이 분해하는 것이다.

$$A = LU$$

- L : 하삼각 행렬 (대각 원소 = 1)
- U : 상삼각 행렬

선형 시스템 $Ax = b$ 는 다음과 같이 변형된다.

$$LUx = b$$

이를 두 단계로 나누어 계산한다.

$$Ly = b \quad (\text{전진 대입})$$

$$Ux = y \quad (\text{후진 대입})$$

2. 3×3 행렬 예시

$$A = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} = LU$$

$$L = \begin{bmatrix} 1 & 0 & 0 \\ l_{21} & 1 & 0 \\ l_{31} & l_{32} & 1 \end{bmatrix}, \quad U = \begin{bmatrix} u_{11} & u_{12} & u_{13} \\ 0 & u_{22} & u_{23} \\ 0 & 0 & u_{33} \end{bmatrix}$$

3. Doolittle 공식

상삼각 행렬 원소 계산:

$$U_{ij} = A_{ij} - \sum_{k=1}^{i-1} L_{ik}U_{kj} \quad (i \leq j)$$

하삼각 행렬 원소 계산:

$$L_{ij} = \frac{A_{ij} - \sum_{k=1}^{j-1} L_{ik}U_{kj}}{U_{jj}} \quad (i > j)$$

4. Doolittle 알고리즘 절차

Step 1. 초기화

- L 의 대각 원소를 1로 설정
- 나머지 원소는 0으로 초기화

Step 2. 각 행에 대해 반복

- 먼저 U 의 행 원소 계산
- 다음으로 L 의 열 원소 계산

Step 3. 선형 시스템 풀이

- $Ly = b$ 전진대입
- $Ux = y$ 후진대입

5. 특징

- 가우스 소거와 동일한 연산 구조
- 한 번 분해하면 여러 개의 b 에 대해 효율적
- 계산 복잡도: $O(n^3)$
- 수치적 안정성을 위해 부분 피벗팅과 함께 사용 가능

3.3.3 $A = LU$ 증명

가우스 소거법을 이용하여 $Ax = b$ 를 풀 때, 행렬 A 를 하삼각 행렬 L 과 상삼각 행렬 U 로 분해할 수 있음을 보이자.

(1) 3×3 행렬에 대한 기본 설정

$$A = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} \quad (3.3.3-1)$$

가우스 소거 과정에서 첫 번째 피벗은 a_{11} 이다.

$$f_{21} = \frac{a_{21}}{a_{11}}, \quad f_{31} = \frac{a_{31}}{a_{11}} \quad (3.3.3-2)$$

이를 이용하여 아래 행을 소거하면 상삼각 행렬이 형성된다.

(2) 첫 번째 소거 행렬

$$E_1 = \begin{bmatrix} 1 & 0 & 0 \\ -f_{21} & 1 & 0 \\ -f_{31} & 0 & 1 \end{bmatrix} \quad (3.3.3-3)$$

$$E_1 A = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ 0 & a'_{22} & a'_{23} \\ 0 & a'_{32} & a'_{33} \end{bmatrix}$$

두 번째 피벗 a'_{22} 에 대해

$$f_{32} = \frac{a'_{32}}{a'_{22}} \quad (3.3.3-4)$$

$$E_2 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & -f_{32} & 1 \end{bmatrix} \quad (3.3.3-5)$$

$$E_2 E_1 A = U = \begin{bmatrix} u_{11} & u_{12} & u_{13} \\ 0 & u_{22} & u_{23} \\ 0 & 0 & u_{33} \end{bmatrix} \quad (3.3.3-6)$$

(3) L 행렬의 구성

$$E_2 E_1 A = U$$

따라서

$$A = E_1^{-1} E_2^{-1} U$$

소거 행렬의 역행렬은 다음과 같다.

$$E_1^{-1} = \begin{bmatrix} 1 & 0 & 0 \\ f_{21} & 1 & 0 \\ f_{31} & 0 & 1 \end{bmatrix}$$

$$E_2^{-1} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & f_{32} & 1 \end{bmatrix}$$

따라서

$$L = E_1^{-1} E_2^{-1} = \begin{bmatrix} 1 & 0 & 0 \\ f_{21} & 1 & 0 \\ f_{31} & f_{32} & 1 \end{bmatrix} \quad (3.3.3-7)$$

결론적으로,

$$A = LU$$

(4) 선형 시스템 풀이 과정

$$Ax = b$$

$$LUx = b$$

먼저

$$Ly = b \quad (\text{전진 대입})$$

다음으로

$$Ux = y \quad (\text{후진 대입})$$

—

(5) 핵심 정리

- 가우스 소거는 소거 행렬의 곱으로 표현 가능
- 소거 행렬의 역행렬을 모으면 하삼각 행렬 L 이 된다
- 최종 상삼각 행렬은 U
- 따라서 모든 정칙 행렬은 $A = LU$ 로 분해 가능
- 해 계산은 전진 대입 + 후진 대입 두 단계로 수행

3.3.4 Doolittle LU 분해법 공식 유도 과정 이용해서 손으로 풀기

① 3×3 행렬 예제

다음 행렬에 대하여 $A = LU$ 분해를 수행하자.

$$A = \begin{bmatrix} 2 & 4 & 2 \\ 4 & 12 & 8 \\ 2 & 6 & 3 \end{bmatrix}$$

Doolittle 방법에서

$$L = \begin{bmatrix} 1 & 0 & 0 \\ l_{21} & 1 & 0 \\ l_{31} & l_{32} & 1 \end{bmatrix}, \quad U = \begin{bmatrix} u_{11} & u_{12} & u_{13} \\ 0 & u_{22} & u_{23} \\ 0 & 0 & u_{33} \end{bmatrix}$$

② 첫 번째 행 계산

첫 행 비교:

$$u_{11} = 2, \quad u_{12} = 4, \quad u_{13} = 2$$

두 번째 행:

$$l_{21} = \frac{4}{2} = 2$$

$$l_{21}u_{12} + u_{22} = 12 \Rightarrow 2(4) + u_{22} = 12 \Rightarrow u_{22} = 4$$

$$l_{21}u_{13} + u_{23} = 8 \Rightarrow 2(2) + u_{23} = 8 \Rightarrow u_{23} = 4$$

③ 세 번째 행 계산

$$l_{31} = \frac{2}{2} = 1$$

$$l_{31}u_{12} + l_{32}u_{22} = 6 \Rightarrow 1(4) + 4l_{32} = 6 \Rightarrow l_{32} = \frac{1}{2}$$

$$l_{31}u_{13} + l_{32}u_{23} + u_{33} = 3 \Rightarrow 1(2) + \frac{1}{2}(4) + u_{33} = 3 \Rightarrow u_{33} = -1$$

④ 최종 LU 분해 결과

$$L = \begin{bmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ 1 & \frac{1}{2} & 1 \end{bmatrix} \quad U = \begin{bmatrix} 2 & 4 & 2 \\ 0 & 4 & 4 \\ 0 & 0 & -1 \end{bmatrix}$$

⑤ 연립방정식 풀이

주어진 시스템

$$Ax = b, \quad b = \begin{bmatrix} 2 \\ 16 \\ 4 \end{bmatrix}$$

1단계: 전진대입 $Ly = b$

$$y_1 = 2$$

$$2y_1 + y_2 = 16 \Rightarrow y_2 = 12$$

$$y_1 + \frac{1}{2}y_2 + y_3 = 4 \Rightarrow y_3 = -4$$

2단계: 후진대입

마찬가지로 $Ux = y$ 를 풀어준다

$$\begin{bmatrix} 2 & 4 & 2 \\ 0 & 4 & 4 \\ 0 & 0 & -1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 2 \\ 12 \\ -4 \end{bmatrix} \quad (3.3.4-6)$$

후진대입을 수행하면,

$$\therefore x_1 = -1, \quad x_2 = -1, \quad x_3 = 4$$

3.3.5 Doolittle LU Python 코드 예제

실습예제 1: https://github.com/seejokim1/NA_with_python/blob/main/chapter03/section_03_03_05_doolittle_lu_problem1.py

실습예제 2: https://github.com/seejokim1/NA_with_python/blob/main/chapter03/section_03_03_05_doolittle_lu_solve.py

```
def doolittle_LU(A):
    n = len(A)
    L = np.zeros((n,n))
    U = np.zeros((n,n))

    for i in range(n):
        L[i][i] = 1

        for j in range(i, n):
            U[i][j] = A[i][j] - sum(L[i][k]*U[k][j]
                                   for k in range(i))

        for j in range(i+1, n):
            L[j][i] = (A[j][i] - sum(L[j][k]*U[k][i]
                                   for k in range(i))) / U[i][i]

    return L, U
```

3.3.6 (3.3.6) Crout LU 분해법

Crout LU 분해의 특징

- 행렬 A 를 $A = LU$ 로 분해
- L : 하삼각행렬 (대각 원소 계산)
- U : 상삼각행렬 (대각 원소는 1로 고정)
- Doolittle 방식과 달리 U 의 대각을 1로 둔다.

1. 기본 공식

$$A = LU$$

(1) 하삼각행렬 L 계산

$$L_{ij} = A_{ij} - \sum_{k=1}^{j-1} L_{ik}U_{kj}, \quad i \geq j$$

(2) 상삼각행렬 U 계산

$$U_{ij} = \frac{1}{L_{ii}} \left(A_{ij} - \sum_{k=1}^{i-1} L_{ik}U_{kj} \right), \quad i < j$$

$$U_{ii} = 1$$

2. Crout LU 알고리즘

1. A 는 $n \times n$ 행렬
2. L 과 U 초기화
3. 열(column) 기준으로 계산
4. 먼저 L 의 해당 열 계산
5. 그 다음 U 의 해당 행 계산
6. 모든 열에 대해 반복

3×3 예제

$$A = \begin{bmatrix} -2 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & -1 & 2 \end{bmatrix}$$

Crout 분해 결과:

$$L = \begin{bmatrix} -2 & 0 & 0 \\ 1 & \frac{1}{2} & 0 \\ 0 & -1 & 4 \end{bmatrix} \quad U = \begin{bmatrix} 1 & -\frac{1}{2} & 0 \\ 0 & 1 & 2 \\ 0 & 0 & 1 \end{bmatrix}$$

선형시스템 풀이

$$Ax = b \Rightarrow L U x = b$$

$$Ly = b \quad (\text{전진대입})$$

$$Ux = y \quad (\text{후진대입})$$

최종 해:

$$x_1 = \frac{13}{2}, \quad x_2 = 15, \quad x_3 = \frac{19}{2}$$

실습예제: https://github.com/seejokim1/NA_with_python/blob/main/chapter03/section_03_03_06_crout_lu_decomposition.py

3.3.7 $[A] = [L][D][L]^T$ 분해 및 Cholesky LU 분해

1. $A = LDL^T$ 분해

대칭 행렬 A 는 다음과 같이 분해할 수 있다.

$$A = LDL^T$$

- L : 하삼각 행렬 (대각 원소 1)
- D : 대각 행렬
- L^T : L 의 전치 행렬

특징

- 대칭 행렬에서 사용 가능
- Cholesky 분해와 유사한 구조
- 계산량 감소 및 수치적 안정성 향상

2. Cholesky LU 분해

대칭 양의 정부호 행렬 A 에 대해,

$$A = LL^T$$

로 분해할 수 있다.

- L : 하삼각 행렬
- L^T : 상삼각 행렬
- D 행렬이 필요 없음

조건

- $A = A^T$
- 모든 고유값이 양수 (Positive Definite)

3. Cholesky 분해 공식

대각 원소 계산:

$$l_{jj} = \sqrt{a_{jj} - \sum_{k=1}^{j-1} l_{jk}^2}$$

비대각 원소 계산:

$$l_{ij} = \frac{1}{l_{jj}} \left(a_{ij} - \sum_{k=1}^{j-1} l_{ik} l_{jk} \right), \quad i > j$$

4. 알고리즘 절차

1. A 가 대칭 양의 정부호인지 확인
2. 첫 번째 대각 원소 계산
3. 첫 번째 열의 나머지 원소 계산
4. 다음 대각 원소 계산
5. 반복 수행

5. 연립방정식 풀이

$$Ax = b$$

Cholesky 분해를 이용하면

$$LL^T x = b$$

1. 전진 대입: $Ly = b$
2. 후진 대입: $L^T x = y$

6. 장점 비교

- 일반 LU 분해보다 계산량 감소
- 수치적으로 더 안정적
- 대규모 대칭 시스템에서 매우 효율적

실습예제: https://github.com/seejokim1/NA_with_python/blob/main/chapter03/section_03_03_07_cholesky_decomposition.py

3.3.8 SciPy에서의 LU 분해

SciPy에서는 `scipy.linalg.lu` 함수를 사용하여 LU 분해를 수행한다.

이 함수는 다음 세 행렬을 반환한다.

$$PA = LU$$

- P : 행 교환 행렬 (Pivoting 행렬)
- L : 하삼각 행렬 (대각 원소 = 1)
- U : 상삼각 행렬

즉, SciPy는 부분 피벗팅을 포함한 LU 분해 결과를 제공한다.

실습예제: https://github.com/seejokim1/NA_with_python/blob/main/chapter03/section_03_03_08_lu_scipy.py

```
from scipy.linalg import lu
P, L, U = lu(A)
```

3.3.9 SciPy에서 LU 분해를 활용한 선형 방정식 풀이

SciPy에서는 `lu_factor`와 `lu_solve` 함수를 사용하여 LU 분해를 기반으로 선형 방정식을 효율적으로 풀 수 있다.

$$Ax = b$$

- `lu_factor(A)` : LU 분해 및 피벗 정보 반환
- `lu_solve((lu, piv), b)` : 해 벡터 x 계산

예제 코드:

```
from scipy.linalg import lu_factor, lu_solve
import numpy as np

A = np.array([[2, 1, -1],
              [-3, -1, 2],
              [-2, 1, 2]], dtype=float)

b = np.array([8, -11, -3], dtype=float)

lu, piv = lu_factor(A)
x = lu_solve((lu, piv), b)

print(x)
```

$$x = [2, 3, -1]$$

실습예제: https://github.com/seejokim1/NA_with_python/blob/main/chapter03/section_03_03_09_lu_solve_scipy.py

3.3.10 LU 분해법과 NumPy의 `numpy.linalg.solve` 비교

NumPy의 `numpy.linalg.solve` 함수는 내부적으로 LU 분해를 사용하여 선형 방정식을 푼다. 하지만 L , U 행렬을 별도로 반환하지 않고 해를 직접 계산한다.

$$Ax = b$$

- `numpy.linalg.solve(A, b)`
- 내부적으로 LU 분해 기반 계산 수행
- 해 벡터만 반환

예제 코드:

```
import numpy as np

A = np.array([[2, 1, -1],
              [-3, -1, 2],
```

```

        [-2, 1, 2]], dtype=float)

b = np.array([8, -11, -3], dtype=float)

x = np.linalg.solve(A, b)

print(x)

```

$$x = [2, 3, -1]$$

실습예제: https://github.com/seejokim1/NA_with_python/blob/main/chapter03/section_03_03_10_lu_vs_numpy_solve.py

```
x = np.linalg.solve(A, b)
```

3.4 Jacobi, Gauss-Seidel, SOR 반복법

3.4.1 Jacobi, Gauss-Seidel, SOR 반복법 개요

연립방정식 $Ax = b$ 의 해법은 직접법과 반복법으로 나뉜다.

- 직접법: 가우스 소거법, LU 분해 등
- 반복법: 초기값에서 시작하여 반복 계산으로 수렴

대규모 희소 행렬(Sparse matrix)에 대해 반복법이 효율적이다.

1. Jacobi 방법

각 반복에서 이전 단계 값만 사용한다.

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j=1, j \neq i}^n a_{ij} x_j^{(k)} \right)$$

특징

- 병렬 계산에 유리
- 수렴 속도는 상대적으로 느림

2. Gauss-Seidel 방법

계산된 최신 값을 즉시 사용한다.

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j=1}^{i-1} a_{ij} x_j^{(k+1)} - \sum_{j=i+1}^n a_{ij} x_j^{(k)} \right)$$

특징

- Jacobi보다 빠르게 수렴
- 병렬화는 어려움

3. SOR (Successive Over-Relaxation)

Gauss-Seidel에 가중치 ω 를 추가.

$$x_i^{(k+1)} = (1 - \omega)x_i^{(k)} + \frac{\omega}{a_{ii}} \left(b_i - \sum_{j=1}^{i-1} a_{ij}x_j^{(k+1)} - \sum_{j=i+1}^n a_{ij}x_j^{(k)} \right)$$

가중치 특성

- $\omega = 1$: Gauss-Seidel
- $\omega > 1$: 가속
- $\omega < 1$: 감속

4. 수렴 조건

행렬 A 가 대각우세(diagonally dominant)일 때 수렴:

$$|a_{ii}| > \sum_{j \neq i} |a_{ij}|$$

또는 대칭 양의 정부호(SPD) 행렬인 경우 수렴한다.

5. 활용

- 대규모 선형 시스템
- CFD, 전산역학
- 희소 행렬 문제

3.4.2 Jacobi 반복법 (핵심 정리)

1. 기본 개념

선형연립방정식

$$A\mathbf{x} = \mathbf{b}$$

에서 각 식을 대각 원소 기준으로 정리하면 Jacobi 반복식은 다음과 같다.

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j=1, j \neq i}^n a_{ij}x_j^{(k)} \right)$$

즉, 모든 미지수를 이전 반복값 $x^{(k)}$ 만을 이용하여 계산한다.

2. 행렬 형태

행렬을 다음과 같이 분해한다.

$$A = D + (A - D)$$

여기서

$$D = \text{대각행렬}, \quad M = A - D$$

Jacobi 반복식은 다음과 같이 표현된다.

$$D\mathbf{x}^{(k+1)} = \mathbf{b} - M\mathbf{x}^{(k)}$$

또는

$$\mathbf{x}^{(k+1)} = D^{-1}(\mathbf{b} - M\mathbf{x}^{(k)})$$

—

3. 알고리즘

1. 초기값 $\mathbf{x}^{(0)}$ 선택
2. 반복 계산:

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j \neq i} a_{ij} x_j^{(k)} \right)$$

3. 수렴 조건 만족 시 종료

—

4. 수렴 조건

Jacobi 방법은 다음 조건에서 수렴한다.

- 대각 우세 행렬 (Diagonally Dominant)

$$|a_{ii}| > \sum_{j \neq i} |a_{ij}|$$

- 또는 대칭 양의 정부호 행렬 (SPD)

—

5. Condition Number

행렬의 민감도는 다음으로 정의된다.

$$K(A) = \|A\| \|A^{-1}\|$$

2-노름 기준에서는

$$K(A) = \frac{\sigma_{\max}}{\sigma_{\min}}$$

여기서 σ 는 특이값(Singular Value)이다.

Condition number가 클수록 수치적으로 불안정하다.

정리

- Jacobi는 병렬 계산에 유리
- 이전 반복값만 사용

실습예제: https://github.com/seejokim1/NA_with_python/blob/main/chapter03/section_03_04_02_jacobi_method.py

```
def jacobi(A, b, x0, tol=1e-6, max_iter=100):
    n = len(b)
    x = x0.copy()

    for _ in range(max_iter):
        x_new = np.zeros(n)
        for i in range(n):
            s = sum(A[i][j]*x[j] for j in range
                    (n) if j != i)
            x_new[i] = (b[i] - s) / A[i][i]

        if np.linalg.norm(x_new - x) < tol:
            return x_new

    x = x_new

    return x
```

3.4.3 Gauss-Seidel 반복법 (핵심 정리)

1. 기본 개념

Gauss-Seidel 방법은 Jacobi 방법의 개선형이다. 차이점은 다음과 같다.

- Jacobi: 이전 반복값만 사용
- Gauss-Seidel: 이미 계산된 최신값을 즉시 사용

2. 반복 공식

선형 시스템

$$Ax = b$$

에 대해 Gauss-Seidel 반복식은

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j=1}^{i-1} a_{ij}x_j^{(k+1)} - \sum_{j=i+1}^n a_{ij}x_j^{(k)} \right)$$

즉, 앞쪽 항은 최신값 $x^{(k+1)}$, 뒤쪽 항은 이전값 $x^{(k)}$ 을 사용한다.

3. 알고리즘

1. 초기값 $\mathbf{x}^{(0)}$ 설정
2. 각 성분을 순차적으로 갱신:

$$x_1^{(k+1)}, x_2^{(k+1)}, \dots, x_n^{(k+1)}$$

3. 수렴 판정:

$$\|\mathbf{x}^{(k+1)} - \mathbf{x}^{(k)}\| < \varepsilon$$

4. 특징

- Jacobi보다 빠른 수렴
- 순차 계산 $\rightarrow$ 병렬화 어려움
- 행 재배열(피벗팅)이 수렴성에 영향
- 대각 우세 행렬에서 수렴 보장

5. 예제 형태

예:

$$\begin{aligned} 4x_1 + x_2 - x_3 &= 3 \\ 2x_1 + 7x_2 + x_3 &= 18 \\ x_1 - 3x_2 + 12x_3 &= 31 \end{aligned}$$

초기값:

$$\mathbf{x}^{(0)} = [0, 0, 0]^T$$

허용 오차:

$$\varepsilon = 10^{-6}$$

정리

- Jacobi보다 수렴 빠름
- 최신값 즉시 반영
- 순서에 따라 수렴 속도 달라짐
- 대규모 희소 행렬에 효과적

실습예제: https://github.com/seejokim1/NA_with_python/blob/main/chapter03/section_03_04_03_gauss_seidel_method.py

```

def gauss_seidel(A, b, x0, tol=1e-6, max_iter
=100):
    n = len(b)
    x = x0.copy()

    for _ in range(max_iter):
        x_old = x.copy()
        for i in range(n):
            s1 = sum(A[i][j]*x[j] for j in
range(i))
            s2 = sum(A[i][j]*x_old[j] for j in
range(i+1, n))
            x[i] = (b[i] - s1 - s2) / A[i][i]

        if np.linalg.norm(x - x_old) < tol:
            return x

    return x

```

3.4.4 SOR (Successive Over-Relaxation) 반복 법

1. 기본 개념

SOR 방법은 Gauss-Seidel 방법에 가중치 ω 를 추가한 방법이다.

- $\omega = 1 \rightarrow$ Gauss-Seidel과 동일
- $\omega > 1 \rightarrow$ 가속(Over-relaxation)
- $0 < \omega < 1 \rightarrow$ 감속(Under-relaxation)

2. 반복 공식

$$x_i^{(k+1)} = (1 - \omega)x_i^{(k)} + \frac{\omega}{a_{ii}} \left(b_i - \sum_{j=1}^{i-1} a_{ij}x_j^{(k+1)} - \sum_{j=i+1}^n a_{ij}x_j^{(k)} \right)$$

3. 수렴 특성

- 최적의 ω 선택 시 수렴 속도 크게 향상
- 대각우세 행렬(diagonally dominant)에서 수렴
- 대칭 양의 정부호(SPD) 행렬에서도 수렴
- ω 값이 부적절하면 발산 가능

4. 정리

- Jacobi < Gauss-Seidel < SOR (수렴 속도)

- 대규모 희소 행렬 문제에 효과적
- ω 선택이 핵심

실습예제: https://github.com/seejokim1/NA_with_python/blob/main/chapter03/section_03_04_04_sor_method.py

```
def sor(A, b, x0, omega, tol=1e-6, max_iter=100):
    n = len(b)
    x = x0.copy()

    for _ in range(max_iter):
        x_old = x.copy()
        for i in range(n):
            s1 = sum(A[i][j]*x[j] for j in range(i))
            s2 = sum(A[i][j]*x_old[j] for j in range(i+1, n))
            x[i] = (1-omega)*x_old[i] + omega*(b[i] - s1 - s2)/A[i][i]

        if np.linalg.norm(x - x_old) < tol:
            return x

    return x
```

3.5 연습문제